Common Baseline for the Vector Database Project

Principles

We align only on what changes a measured number or decides whether one group can run another group's project. Everything else stays free.

Common to all groups:

- the dataset files, test queries, k values and distance
- resource limits and the timing method
- the id and vector columns, and the order of loading steps
- the form of the CLI and of the result files
- the clustering algorithm and its parameters

Free for each group:

- the database, language and libraries
- the index type and its parameters
- extra columns, internal design and extra experiments

1. Datasets

Everyone benchmarks on the same unmodified dataset files and uses the ground truth that ships with them. Sizes are raw float32 vectors (rows x dimension x 4 bytes).

Dimension	Size	Dataset	Rows x dim	Size (GB)	Distance	Others	Disk needed (about 3x, GB)
128 to 256	S	VIBE glove-200-cosine	1,192,514 x 200	0.95	cosine	in-distribution	3
128 to 256	M	?					
128 to 256	L	?					
768	S	VIBE landmark-nomic-768-normalized	760,757 x 768	2.3	any (normalized), use cosine	in-distribution	7
768	M	VIBE arxiv-nomic-768-normalized	1,344,643 x 768	4.1	any (normalized), use cosine	in-distribution	12

Dimension	Size	Dataset	Rows x dim	Size (GB)	Distance	Others	Disk needed (about 3x, GB)
768	L	VIBE hotpotqa-harrier-640-normalized	5,233,329 x 640	13.4	any (normalized), use cosine	out-of-distribution	40
2048	S	VIBE celeba-resnet-2048-cosine (deprecated)	201,599 x 2048	1.65	cosine	In-distribution, deprecated	5
2048	M	VIBE inaturalist-resnet-2048-cosine	499,000 x 2048	4.1	cosine	in-distribution	12
2048	L	?					

- All the chosen datasets have ground truth computed. If we choose a part of the dataset, we would need to compute the ground-truth for selected rows - for example if we want to measure recall later
- Data type is the stored number format (VIBE files carry it in the `point_type` attribute, so confirm it in each file).
- Distance is the one the ground truth was computed with.
- Celeba is deprecated, which VIBE says only means its benchmark results are no longer updated; the file stays available.

Rules for every dataset:

1. Use the file's own `train` (data), `test` (queries) and `neighbors` (true neighbors). Do not subsample or normalize; the only allowed conversion is uint8 to float32.
2. Queries are the `test` vectors, with k = 10 and k = 100 (the files hold 100 neighbors).
3. Distance is the one named in the file. If the file says "any" (normalized vectors), use cosine, the metric of most datasets in the table.
4. Ground-truth ids are 0-based row positions in `train`, so the id column must equal that position.
5. A download step fetches the file, prints its size and checksum.

2. Deployment

Each group ships one Dockerfile and runs it under the same resource limits, so differences in results come from the database and not from the machine.

Setting	Proposal	Reason
Runtime	Docker or Podman, same flags in both	Either works for everyone.

Setting	Proposal	Reason
CPU	4 cores, pinned with `--cpuset-cpus`	Pinning reduces scheduler noise.
RAM	TBD, swap off (`--memory-swap` equals `--memory`)	Should fit an in-memory index for all datasets, so no system is excluded. The 13.4 GB dataset would need roughly 15 to 18 GB - not feasible on some of the student's laptops, I would suggest to drop it
Storage	SSD, TBD, about 3× the largest dataset in the table)	Disk-based indexes depend on disk speed.
Paths in the container	`/data` (read-only, except during the download step), `/db`, `/results`	Anyone can run any project without reading its code.
Versions	Database, language and libraries pinned in the repo	Behavior changes between versions.

Embedded systems (LanceDB, DuckDB, Chroma in local mode) run the database and the benchmark code in one container with these limits. Server systems (PostgreSQL, Qdrant, Milvus) run the database in a container with the same limits, and the client in a second container with its own core. Each group's README gives the exact commands: build, download the dataset, and run.

```
docker compose build
docker compose run --rm download
docker compose run --rm benchmark db 1 443321 4
```

Resource limits for laptops

The limits are chosen so that a laptop never swaps, never throttles, and never has to read data from the SSD during warm queries. Otherwise the benchmark measures the laptop and not the database.

Upper bounds for a laptop with 16 GB RAM and about 200 GB disk

Resource	Upper bound for the experiment	Why
RAM	Docker VM at most 10 GB, container at most 8 GB, swap off	macOS and open applications need about 4 to 6 GB. If the VM takes more, the host starts swapping and timings become meaningless.

CPU	At most 4 cores, and at most half of the laptop's cores	The host needs spare cores. Using all of them causes throttling.
Disk	At most 100 GB for data, databases and results, with at least 50 GB kept free	SSDs slow down when nearly full, and macOS needs free space for its own use.
During a run	Laptop plugged in, other heavy apps closed, memory pressure green	Reduces noise that comes from the laptop.

Rule for including a dataset: **raw size x 1.5 must not exceed the container RAM limit** (at 8 GB, raw size up to about 5.3 GB). A larger dataset can't be held by an in-memory index, and a disk-based index would read from a cold SSD, so the result would measure the laptop's disk.

How big are the indexes, and why

- **HNSW (graph index). I**ndex size is generally comparable to the raw vector data itself, with the graph structure adding a relatively modest overhead (typically about 5% to 20% extra, depending on the configuration - connections per node - and engine)
- **IVF-PQ (LanceDB).** Per vector, the index stores M one-byte codes plus a row id, which I assume is ~8bytes. The size is about rows x (M + 8), plus a few MB of centroids and codebooks. With M = dim / 8, that is a 24x to 31x compression of what is searched. The table with the full vectors still stays on disk for re-ranking, so the database size is about raw + index. Only the compact index must stay in memory.
- **No index (exact search).** No extra size, but every query reads the whole table. It is useful as a baseline.
- **RAM rule of thumb for in-memory HNSW:** 1.1 to 1.5 x raw size. The 1.0 is the vectors, 0.02 to 0.17 is the graph, and the rest is build overhead, copies and the allocator.
- **Disk planning:** about 3 x raw size at peak: the downloaded file (1x), the database copy (1x), the index (0.04 to 1.2x), and temporary space for builds, sorting and old versions.

Verdict per dataset (8 GB container)

Dataset	Raw	HNSW RAM	IVF-PQ index (M = dim / 8)	Peak disk	Verdict
glove-200	0.95 GB	1.0 to 1.4 GB	39 MB (M = 25)	2.9 GB	Accept
celeba-resnet-2048	1.65 GB	1.8 to 2.5 GB	53 MB (M = 256)	5.0 GB	Accept

landmark-nomic-768	2.34 GB	2.6 to 3.5 GB	79 MB (M = 96)	7.0 GB	Accept
arxiv-nomic-768	4.13 GB	4.5 to 6.2 GB	140 MB (M = 96)	12.4 GB	Accept, tight for HNSW
inaturalist-resnet-2048	4.09 GB	4.5 to 6.1 GB	132 MB (M = 256)	12.3 GB	Accept, tight for HNSW
hotpotqa-harrier-640	**13.4 GB**	**14.7 to 20.1 GB**	461 MB (M = 80)	**40.2 GB**	**Reject**

3. Measurements

For every dataset and every k, each group reports recall, latency percentiles, loading and index times, disk size and peak memory, measured the same way.

Quantity	Definition
Recall@k (k = 10 and 100)	Overlap between the returned ids and the file's `neighbors`, divided by k, averaged over all test queries
Latency (ms)	p50, p95 and p99 per query, measured by the client around the search call, one query at a time, one thread, returning ids only
Ingestion time (s)	From the start of loading to the last stored row, excluding the download
Index build time (s)	At most 4 threads
Index size and total database size (MB)	Measured on disk after loading and indexing are finished
Peak memory (MB)	Peak resident memory (or container memory peak) during loading, index build and queries
Throughput (queries per second)	Optional, with 4 concurrent clients
Clustering time(s)	Total time of one clustering run with the agreed parameters

Protocol. Run 200 warm-up queries and discard them, then run 3 passes over all test queries and report p50, p95 and p99 over all samples. Report warm numbers. A cold run is optional, and if given, state how caches were cleared.

Accuracy settings. Every system has a search parameter that trades recall for speed (for example `nprobes` or `ef_search`). Report at least 5 settings that span recall@10 from about 0.8 to 0.99. The headline number for comparing groups is the latency at the lowest setting that reaches recall@10 of at least 0.95.

Log with every result: CPU model, cores, RAM, disk type, OS, database and library versions, parameters used, and the date.

4. Clustering

Besides KNN, each group also runs K-means (or another clustering algorithm) on at least one dataset. The distance is the one used for the dataset, and the number of clusters, the iteration limit, the initialization and the random seed are the same for all groups. Their values are set together with the goals in the next classes. A group states whether the clustering runs inside the database or outside it (for example with scikit-learn or Faiss), and stores the result as a cluster id per vector. Where it is stored is the group's own design. Report the running time, peak memory and the cluster sizes.

5. Database design

Every group stores each dataset in one table (or collection) with two required columns: an integer id equal to the row position and a float32 vector.

Column	Type	Meaning
`id`	64-bit integer	0-based row position in `train`; must match the ids in `neighbors`
`vector`	float32, fixed length d	The embedding, with d taken from the dataset

Nothing else is required. Extra columns (metadata, cluster ids from K-means) are each group's own design and will be settled together with the goals in the next topic. The table is named after the dataset file, without the extension and with `-` replaced by `_` (for example `glove_200_cosine`).

How to document the DDL. Systems define structure in different ways, so the requirement is one place in the repo that shows it, in two marked parts:

- Schema part, run before loading: the table and its two required columns.
- Index part, run after loading: the index with its type and every parameter.

SQL systems use a `.sql` file. Other systems use one script with the same two parts, plus a table in the README with the columns and types above.

If a system cannot create the index after loading (for example it builds the index while data is inserted), say so, put the index definition in the schema part, and report loading and indexing as one ingestion time with the index build time left empty.

6. Populating

Everyone loads the unmodified dataset in the same order: schema, bulk load, index, checks, with loading and indexing timed separately.

1. **Download.** Run the download step (docker compose run --rm download), which fetches the dataset file into /data and prints its size and checksum. All other steps mount /data read-only
2. **Create the schema.** Run the DDL script from the Database design section. Re-running it drops and recreates the table.
3. **Load.** Insert all `train` vectors in batches (about 50,000 rows each), so the whole file never has to fit in memory. Record the ingestion time and peak memory.
4. **Build the index.** Create it after loading, never before, with the documented parameters. Record build time and index size. If the index cannot be built separately, see the DDL exception above.
5. **Check.** The row count equals the number of `train` rows, the dimension matches, and fetching about 10 random ids returns the matching vectors from the file.

The queries and the ground truth are read from the dataset file at benchmark time. They are not stored in the database.

No preprocessing other than uint8 to float32 conversion. If a system needs a different internal format, that conversion is part of loading and is included in the ingestion time.

7. Tooling

Three shared conventions are enough: a repository layout, the CLI form from the assignment, and one result file format.

Language and libraries. Free. Python is the easy choice, because the VIBE files are HDF5 and `h5py` reads them, but it is not required.

Repository layout.

- `README.md` with the documentation checklist from the assignment
- `Dockerfile` (and a compose file only if the system needs one)
- `scripts/` with download, load, index and benchmark scripts
- `results/` with the small result files (committed)
- `data/` and the database directory are never committed

CLI. `db <goal> <params...>`, as in the assignment (for example `db 1 443321 4`). A goal prints one result per line as `id distance`, nearest first. A failure returns a non-zero exit code. The numbered goals themselves are defined in the next topic.

Result files. Two CSV files, one row each, with these columns:

File	Columns
`build.csv`	group, system, version, dataset, ingest_s, build_s, index_mb, db_mb, peak_mem_mb, cpu, cores, ram_gb, date, kmeans_s
`search.csv`	group, system, version, dataset, k, setting_name, setting_value, recall, p50_ms, p95_ms, p99_ms, qps (optional), peak_mem_mb

Every group adds its files to one shared place chosen by the class, so the results can be put in one table. Fix random seeds, and write the exact commands used into the README so anyone can repeat the run.